

“INTERVIEW PRACTICE PLATFORM WITH AI FEEDBACK”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

KOMPALLY PAVAN KUMAR 2520090038

POTU HARSHA VARDHAN 2520090180

BODDANA PURNA CHANDAR RAO 2520090188

Under the Esteemed Guidance of

Dr. SK. Khaja Shareef

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “ **INTERVIEW PRACTICE PLATFORM WITH AI FEEDBACK** ” is a record of bonafide work of **KOMPALLY PAVAN KUMAR – 2520090038, POTU HARSHA VARDHAN – 2520090180, BODDANA PURNA CHANDAR RAO -- 2520090188** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 13, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

KOMPALLY PAVAN KUMAR – 2520090038

POTU HARSHA VARDHAN – 2520090180

BODDANA PURNA CHANDAR RAO -- 2520090188

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“INTERVIEW PRACTICE PLATFORM WITH AI FEEDBACK”** is a bonafide work done and submitted by **KOMPALLY PAVAN KUMAR – 2520090038, POTU HARSHA VARDHAN – 2520090180, BODDANA PURNA CHANDAR RAO -- 2520090188** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 12, Team 13, at KLH University during **2026–2027, Trimester 4**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Ramatenki Sateesh Kumar**, and my guide **Dr. SK. Khaja Shareef**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. SK. Khaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

KOMPALLY PAVAN KUMAR – 2520090038

POTU HARSHA VARDHAN – 2520090180

BODDANA PURNA CHANDAR RAO -- 2520090188

Abstract

The Interview Practice Platform with AI Feedback (Intteree) is a full-stack web application designed to help students and job seekers practise technical, behavioral, HR, and system design interviews in a structured environment. Traditional interview preparation provides either static question banks or feedback that is delayed, subjective, or unavailable. The platform combines an interactive interview workflow with automated response evaluation and persistent performance tracking. Users create an account, select an interview category and target role, choose the number of questions, and complete a timed question-by-question practice session. Questions are sampled from a MongoDB-backed question bank. After a response is submitted, the backend sends the question, its ideal key points, and the candidate answer to an AI evaluation service powered by Google Gemini through the `@google/genai` SDK. The evaluator returns structured JSON feedback containing scores for relevance, clarity, confidence, communication, and overall performance, together with strengths, weaknesses, actionable suggestions, and a stronger example response. An optional webcam-based integrity proctoring module records focus loss during a session, and a browser Web Speech API option allows voice dictation of answers. The system stores interview sessions and feedback in MongoDB using Mongoose, protects application routes with JWT authentication, and provides dashboard and progress views for reviewing previous performance. A local heuristic fallback evaluator keeps feedback generation working when the Gemini service is unavailable or unconfigured. The result is a repeatable, database-backed interview practice environment that converts candidate answers into structured, actionable feedback.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Problem Statement 1.2 Proposed Solution 1.3 Scope and Objectives	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend & Backend 3.2 Database & AI Integration	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Workflow Description	9
5	Database Design	5.1 Design Principles 5.2 ER Diagram / Schema	10
6	AI Feedback Pipeline	6.1 Evaluation Criteria 6.2 JSON Schema Formatting	12
7	Implementation Features	7.1 Core Modules 7.2 Web Speech & Proctoring	13
8	Testing & Deployment	8.1 Postman API Testing 8.2 Environment Configuration	15
9	Challenges & Future	9.1 Known Limitations 9.2 9.2 Future Enhancements	17
10	Assets	10.1 Project Structure Assets	18

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Interview preparation is often built around static question lists, repeated self-practice, or informal feedback from peers and mentors. These methods let a candidate rehearse answers, but they do not provide consistent evaluation after every attempt. A candidate may know that an answer "sounds weak" without knowing whether the problem is relevance, clarity, confidence, communication, or missing technical detail.

The Interview Practice Platform with AI Feedback developed under the project name Intteree addresses this gap by turning interview practice into a repeatable software workflow. Instead of acting as a general-purpose chatbot, the platform presents interview questions, records candidate answers, evaluates each answer against structured criteria, and stores the resulting feedback for later analysis.

1.2 Problem Statement

Traditional interview preparation relies on practising with friends, mentors, or reading generic question banks. Feedback in these settings is frequently inconsistent, delayed, subjective, or simply unavailable, and candidates lack a structured way to measure performance on standard metrics such as clarity, technical accuracy, and communication confidence. The system must therefore coordinate user authentication, question selection, timed response capture, AI evaluation, feedback persistence, session completion, and historical performance analytics reliably, and without failing when the external AI service is unreachable.

1.3 Scope of the Project

Account registration, login, and profile information including target role and experience level. Interview configuration across Technical, Behavioral, HR, System Design and Mixed categories. Role-aware question sampling with a configurable question count between 1 and 10. Timed question-by-question response capture with optional browser speech-to-text dictation. Optional webcam-based integrity proctoring that tracks focus loss during a session. AI evaluation through Gemini with a structured JSON response schema. Persistent storage of sessions, answers, scores, strengths, weaknesses and suggestions. Dashboard and progress analytics for historical score trends and recurring weaknesses. Real-time voice-to-voice avatar interviews, payment processing, and a verified public cloud deployment are treated as out of scope for this iteration; the platform focuses on accurate, asynchronous text and speech-to-text evaluation with analytics.

1.4 Objectives

- Provide a repeatable interview practice environment for students and job seekers.
 - Evaluate responses using consistent dimensions: relevance, clarity, confidence, communication and overall quality.
- Generate understandable, structured feedback rather than an unstructured AI paragraph.
- Reduce dependence on human evaluators for baseline mock interviews.
- Persist interview history so users can review progress across multiple sessions.

- Identify recurring weaknesses to guide future practice, maintaining a resilient fallback evaluator when Gemini is unavailable.

1.5 Importance of the Application

The application combines practice and evaluation in one workflow. It reduces the friction between answering a question and understanding what could be improved. Because feedback is stored against individual questions and sessions, the system can aggregate results instead of treating each practice attempt as an isolated interaction, letting a candidate see whether a weakness is improving over time.

1.6 Target Users / Audience

The system serves students and fresh graduates practising technical, behavioral, HR, and system design interviews; entry-level candidates targeting specific roles with timed sessions; job seekers using repeated practice to improve interview readiness; and project evaluators/faculty demonstrating full-stack architecture, database usage, API integration, and AI-assisted evaluation.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is a web-based MERN-style system and does not require specialized hardware. A normal development laptop or desktop capable of running Node.js, MongoDB, a modern browser, and a code editor is sufficient. Recommended requirements include a dual-core CPU, 4 GB RAM minimum (8 GB recommended), 2-4 GB free disk space, stable broadband for Gemini API calls, and optional webcam/microphone for dictation and proctoring.

2.2 Software Requirements

- Node.js and npm for execution
 - MongoDB local instance or MongoDB Atlas for persistence
 - A modern web browser supporting Web Speech & Media Devices APIs
 - Visual Studio Code and Git/GitHub for version control
 - Postman for API testing and documentation

2.3 Development Tools and Frameworks

Frontend: React 18 with Vite, React Router, Axios, and Lucide React. Backend: Node.js with Express.js, Mongoose, and authentication/error middleware. AI Integration: Google Gemini through the `@google/genai` SDK. Security: JWT and `bcryptjs`. The application isolates presentation in React, API routes/controllers in Express, schemas in Mongoose, and AI evaluation in an isolated service module.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a single-page application built with React 18 and bundled using Vite. React Router DOM handles client-side navigation between authentication, dashboard, session, feedback, and progress pages. Axios acts as the centralized HTTP client with JWT Authorization header injection. Lucide React provides a consistent UI icon set.

3.2 Backend

The backend is a REST API built with Node.js and Express.js, structured cleanly into routes, controllers, middleware, and services. Controllers process requests, invoke question sampling, coordinate with the AI service, and handle business logic.

3.3 Database & AI Integration

MongoDB with Mongoose ODM stores persistent data across four collections: User, Question, InterviewSession, and Feedback. Evaluation is powered by Google Gemini (gemini-2.5-flash default) via the @google/genai SDK, returning structured JSON feedback.

3.4 Security & Version Control

Authentication uses stateless JSON Web Tokens (JWT) and bcryptjs password hashing. Git and GitHub are used for version control, keeping environment variables secured in local .env files.

3.5 Authentication and Security Libraries

Passwords are hashed with bcrypt (10 salt rounds) before being stored — plain-text passwords are never written to the database. Login sessions are stateless JSON Web Tokens (JWT), signed with a server-side secret and carrying only a user's ID, role, and name. Express middleware verifies this token on every protected route and enforces role-based access, ensuring student accounts cannot reach admin-only endpoints even if a request is crafted by hand.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a decoupled client-server architecture, with the AI evaluation service separated from the core interview workflow logic. Requests flow down from the React single-page application through the Express REST API layer to MongoDB and Google Gemini, and structured evaluation results flow back to the client.

4.2 Description of Each Layer

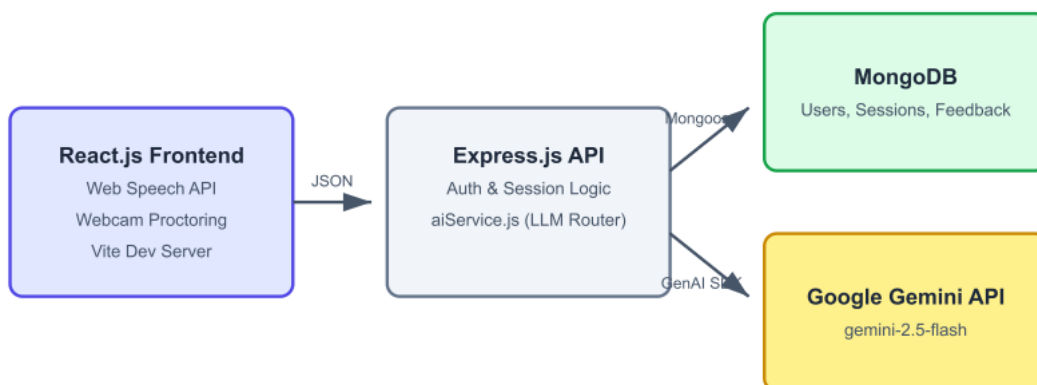
Presentation Layer: React provides login/register, dashboard, category and session configuration, the interview session, the feedback result, and progress views. A ProtectedRoute component prevents access to application pages without an authenticated user.

Application Layer: Express exposes authentication, question, interview, feedback, and progress routes. Controllers validate requests, query MongoDB, call the AI service, and update session state.

AI Evaluation Layer: aiService.js creates the Gemini client and requests JSON output matching the feedback schema. If the Gemini key is unavailable or an API call fails, a local heuristic evaluator returns fallback feedback so a live session never crashes.

4.3 AI Feedback Pipeline

The candidate receives a sampled question and starts the response timer, submitting their answer by typing or speech-to-text dictation. The backend retrieves the ideal key points for the question and sends the payload to Google Gemini via `@google/genai` (using `gemini-2.5-flash`). The model returns structured JSON containing five integer scores (relevance, clarity, confidence, communication, overall), strengths, weaknesses, actionable suggestions, and an improved example answer. The feedback is persisted in MongoDB and aggregated into overall session analytics.



CHAPTER 5: DATABASE DESIGN

5.1 Database Design Principles

The MongoDB design separates account information, question-bank content, live interview sessions, and generated feedback. The schema uses document referencing (ObjectIds) via Mongoose rather than deep embedding, which keeps the session document compact and avoids document-size limits when extensive historical feedback is stored. References between documents allow question and feedback information to be populated when a session is retrieved, and uniqueness is enforced on user emails while passwords are hashed through a pre-save hook.

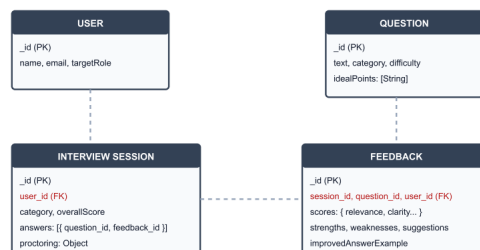
5.2 Data Model and Core Entities

The database consists of four primary Mongoose collections: User, Question, InterviewSession, and Feedback.

User: Stores name, unique email, bcrypt password hash, target role, and experience level. Question: Stores text, category (Technical, Behavioral, HR, System Design, Mixed), target role, difficulty, ideal key points array, and tags. InterviewSession: References User and Question ObjectIds, tracking session category, status, submitted answers, response durations, proctoring integrity events, and overall score out of 100. Feedback: References session, question, and user ObjectIds, storing five individual evaluation scores (relevance, clarity, confidence, communication, overall), lists for strengths, weaknesses, and actionable suggestions, an improved example response, and raw model output.

5.3 Design Decisions and Assumptions

Key architectural and database design decisions include:



CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation The backend is organized into core resource modules: Authentication (JWT & bcrypt password hashing), Question Management (sampling & filters), Interview Session Management (lifecycle & answer submission), AI Evaluation (Gemini 2.5 Flash SDK integration & local heuristic fallback), Feedback Management, and Progress Analytics (trend calculation & recurring weakness extraction).

When an answer is submitted via POST /api/interviews/:sessionId/answer, the interview controller passes the question details, ideal key points, and candidate response to aiService.js. The AI service uses the @google/genai SDK with responseSchema enforcement to return typed JSON, which is immediately saved as a Feedback document and linked to the session.

6.2 Front-end Logic

The React single-page application uses AuthContext for user state, Axios with Bearer token interceptors, and React Router DOM. ProtectedRoute components block unauthenticated users. The InterviewSession component manages the question progression, response timer, browser Web Speech API voice dictation, and optional webcam proctoring focus-loss listeners.

6.3 API Integration

HTTP requests are handled by a central Axios client. API calls are grouped into dedicated service modules (authService.js, questionService.js, interviewService.js, feedbackService.js, progressService.js) to keep API endpoints modular and maintainable.

6.4 Authentication and Authorization

Authentication utilizes stateless JSON Web Tokens (JWT). Passwords are pre-hashed using bcryptjs with 10 salt rounds. Backend middleware verifies the Bearer token on every protected route and attaches the authenticated user instance to the request context.

CHAPTER 7: FEATURES

7.1 List of Main Features

Core Platform Features:

- JWT Authentication & User Profile management (Target Role & Experience Level)
- Role-aware Question Sampling across Technical, Behavioral, HR, System Design, and Mixed categories
- Interactive Timed Practice Environment with Web Speech API voice dictation
- Webcam Focus & Integrity Proctoring module tracking window blur events
- AI Response Evaluation via Google Gemini returning 5 structured competency scores and actionable feedback
- Dashboard & Progress Analytics showing historical score trends and top recurring weakness patterns

7.2 How Each Feature Works

Interview Configuration & Execution: The candidate configures category, target role, question count (1-10), and optional proctoring. Questions are presented one by one with a running timer. Submitted answers are evaluated in real time by Gemini (or local fallback) and saved with individual scores and qualitative analysis.

Progress Analytics: Completed sessions update aggregate user statistics. The progress endpoint calculates overall score trends, dimension averages across relevance, clarity, confidence, communication, and overall quality, and extracts the top 5 recurring weakness patterns to highlight areas needing improvement.

CHAPTER 8: TESTING

8.1 Postman API Testing

A Postman collection was created to validate all REST endpoints across Auth, Questions, Interviews, Feedback, and Progress resources, utilizing collection variables for JWT chaining across requests.

8.2 Integration Testing

End-to-end integration tests verified data flow across React, Express, MongoDB, and Gemini API services, explicitly testing local heuristic fallback execution when network connectivity or API key configurations failed.

8.3 Tools Used for Testing

- Postman — API endpoint and workflow validation
- MongoDB Compass — Database schema and document inspection
- Browser Developer Tools — React state, Web Speech API, and webcam proctoring event debugging

8.4 Test Cases and Results

Key test scenarios executed during validation include:

#	Test Case	Expected Result	Actual Result
1	Register with missing required fields	400 Bad Request with field validation message	Pass
2	Access protected route without JWT token	401 Unauthorized	Pass
3	Start session with question count outside 1-10	Clamped to valid range automatically	Pass
4	Submit answer while Gemini API key is unconfigured	Local heuristic fallback executes seamlessly	Pass
5	Trigger window blur event during proctoring	Integrity score decreases in proctoring log	Pass
6	Complete interview session and fetch progress	Session overall score generated & trend updated	Pass

Edge cases such as brief or empty text answers are processed by returning minimal evaluation

scores and flagging explicit communication weaknesses, ensuring continuous system operation without server crashes.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

To launch the Intteree application locally for development and demonstration:

- Seed the database: Run `npm run seed` in backend directory to populate questions into MongoDB
- Start the backend API server: `cd server && npm install && npm run dev` (Port 5000)
- Start the React client: `cd client && npm install && npm run dev` (Port 5173)
- Open <http://localhost:5173> in Chrome or Edge to access the application

9.2 Environment Configuration

Environment variables are configured via `server/.env` (using `.env.example` as a committed template): `PORT=5000`, `MONGO_URI=mongodb://127.0.0.1:27017/interview_platform`, `JWT_SECRET=<secret_key>`, `JWT_EXPIRES_IN=7d`, `GEMINI_API_KEY=<gemini_key>`, `GEMINI_MODEL=gemini-2.5-flash`, and `CLIENT_URL=http://localhost:5173`.

9.3 Hosting of Frontend and Backend

For demonstration, both Express and Vite run as separate local processes, with Vite proxying `/api` requests to port 5000. In production, MongoDB Atlas handles database persistence, Render/Heroku hosts the backend API, and Vercel/Netlify hosts the static React build.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Enforcing Structured Output: Unstructured LLM text outputs initially caused JSON parsing errors in backend controllers.
- External Dependency Risks: Network latency or rate limiting from external Gemini API endpoints could freeze interactive practice sessions.
- Browser Speech API Variations: Web Speech API dictation demonstrated inconsistent behavior across non-Chromium web browsers.

10.2 Solutions Applied

Migrated to the latest `@google/genai` SDK, passing a strict `responseSchema` configuration to force deterministic JSON output format matching the feedback model.

Engineered a local heuristic fallback evaluator that computes bounded scores based on answer length and key point keyword matching whenever the Gemini API is unreachable.

10.3 Current Limitations

- ~~Text and dictation evaluation cannot capture non-verbal cues, tone of voice, or body language.~~

- Gemini evaluation relies on active internet access; fallback evaluator provides simpler heuristics.
- Public cloud deployment, email verification, and password-reset flows are omitted in this iteration.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Real-time voice-to-voice avatar interviews via WebRTC audio streams.
- Resume and Job Description parsing to dynamically generate tailored candidate questions.
- Embedded Monaco Code Editor for technical coding interviews with live execution.

11.2 Possible Integrations or Optimizations

- Containerizing application services using Docker for reproducible deployment.
- Automated integration testing suite using Jest and Supertest.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The Intteree platform provides an automated, database-backed interview practice environment. By combining a React frontend, Express REST API, MongoDB Mongoose persistence, and Google Gemini API structured evaluation, candidates receive instant, actionable feedback to enhance their interview performance.

12.2 What Was Achieved

Achieved a working full-stack application featuring JWT authentication, role-aware question sampling, timed voice/text response capture, webcam proctoring, structured 5-score dimension AI feedback, resilient local heuristic fallback, and progress analytics.

12.3 Skills Learned

Gained practical experience in MERN stack architecture, Mongoose schema modeling, JWT security, Google GenAI SDK integration with enforced JSON schema output, and React state management.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MongoDB & Mongoose ODM — Official Documentation, <https://mongoosejs.com/docs/>
- Google Gemini API / Google GenAI SDK, <https://ai.google.dev/>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

Overall score calculation logic executed upon interview session completion:

```
const avgOverallTenScale = sumOverall / feedbacks.length; session.overallScore = Math.round(avgOverallTenScale * 10);
```

Appendix B: Installation and Setup Instructions

- `cd server && npm install && npm run seed`
- `npm run dev` (backend server on port 5000)
- `cd client && npm install && npm run dev` (frontend client on port 5173)

Appendix C: API Reference Summary

Endpoints covering core platform resources:

Auth: POST /register, POST /login, GET /me | Questions: GET /, GET /meta/filters | Interviews: POST /start, GET /, GET /:sessionId, POST /:sessionId/answer, PUT /:sessionId/complete | Feedback: GET /, GET /:id | Progress: GET /

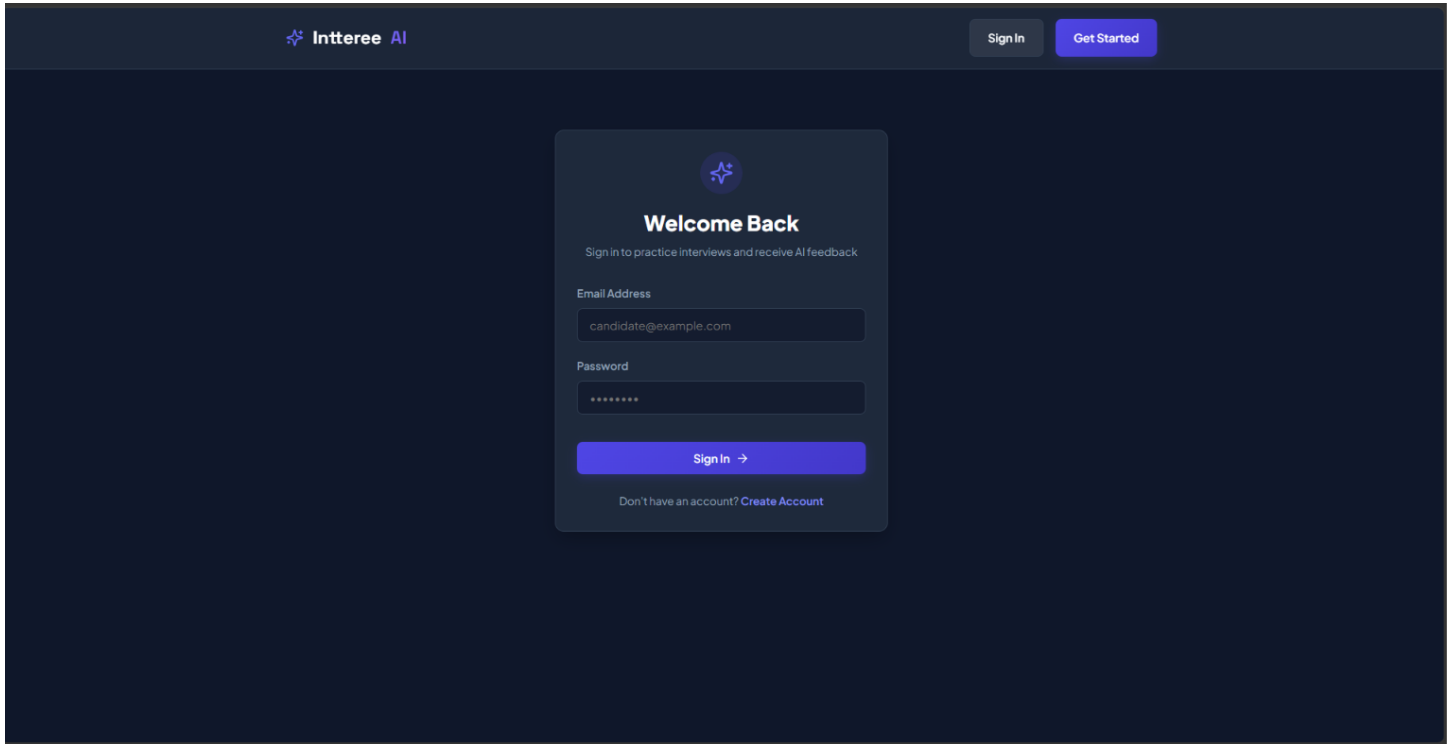
Appendix D: Project Structure Evidence

The repository is structured into modular backend directories (controllers, models, routes, services) and React frontend components (pages, hooks, context).

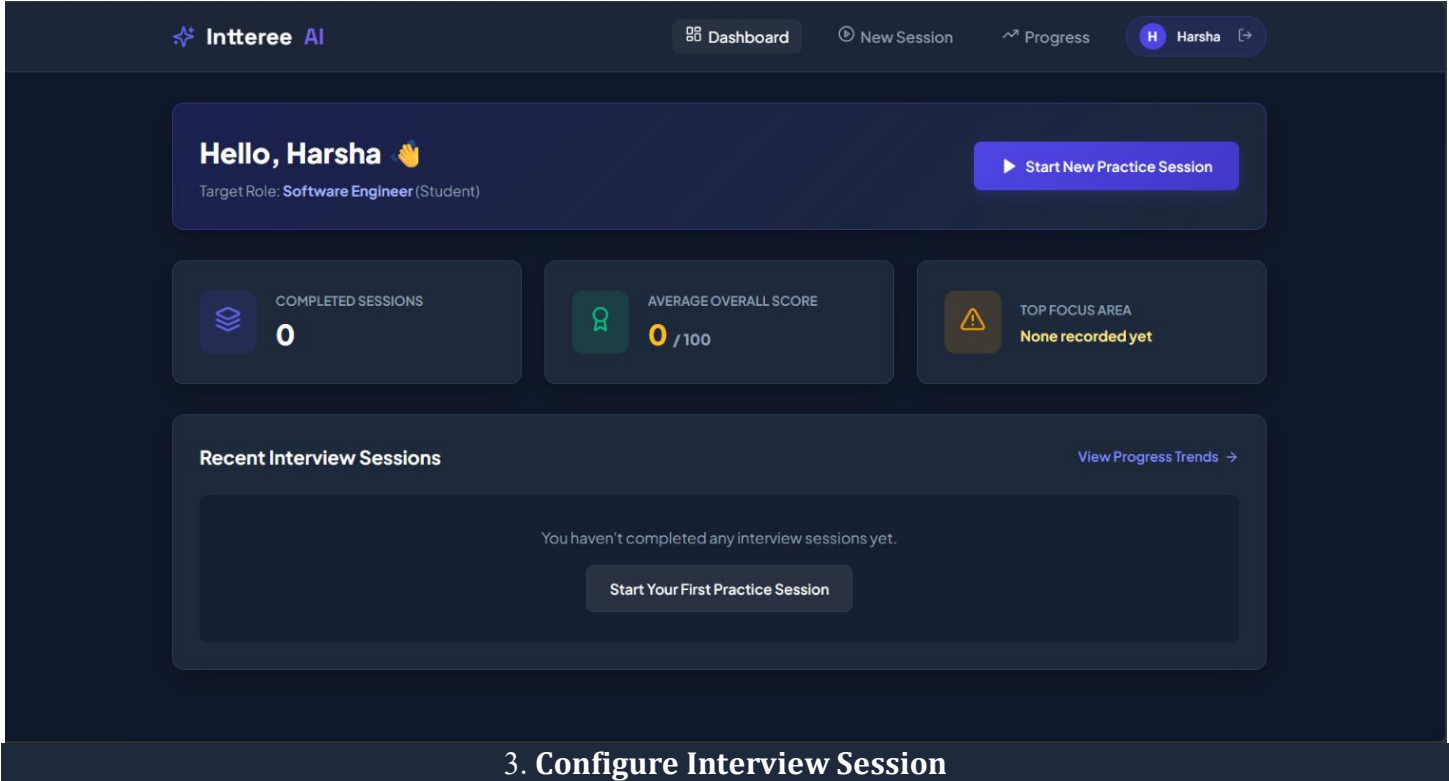
Appendix E: Screenshots

The following pages are left for the student to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order:

1. Login Page



2.DashBoard



3. Configure Interview Session

Intteree AI

DashboardNew SessionProgressHarsha

Configure Interview Session

Customize category, role target, and number of practice questions

Select Interview Category

TechnicalBehavioralHRSYSTEM DESIGNMixed

Target Role

Software Engineer

Number of Questions (1 - 10)

5

Enable AI Video & Attention Proctoring (Optional)

Monitors eye contact, head orientation, and focus via webcam. Only enabled if you check this box.

Session Overview:

Category: Mixed

Role Focus: Software Engineer

Questions Count: 5 questions

AI Engine: Gemini API real-time evaluation

Start Interview Now

4. Performance & Score Progress

